To fine-tune safety, you can explicitly remove the taintedness on Strings of Code before you evaluate them (you can do that by calling Object#untaint) and then rely on safe levels to disallow dangerous operations such as disk access.

By using safe levels carefully, you can write a controlled environment for eval. Such an environment is called a *Sandbox*. Let's take a look at a Sandbox taken from a real-life library.

Spell: Sandbox

The ERB Example

The ERB standard library is the default Ruby template system. This library is a *Code Processor (144)* that you can use to embed Ruby into any file, such as this template containing a snippet of HTML:

ctwc/template.rhtml
```
<p><strong>Wake up!</strong> It's a nice sunny <%= Time.new.strftime("%A") %>.</p>
```

The special <%= ... > tag contains embedded Ruby code. When you pass this template through ERB, the code is evaluated:

ctwc/erb_example.rb
```
require 'erb'
erb = ERB.new(File.read('template.rhtml'))
erb.run
```

```
<p><strong>Wake up!</strong> It's a nice sunny Friday.</p>
```

Somewhere in ERB's source, there must be a method that takes a snippet of Ruby code extracted from the template and passes it to eval. Sure enough, here it is:

```
class ERB
  def result(b=new_toplevel)
    if @safe_level
      proc {
        $SAFE = @safe_level
        eval(@src, b, (@filename || '(erb)'), 0)
      }.call
    else
      eval(@src, b, (@filename || '(erb)'), 0)
    end
  end
  #...
```

new_toplevel is a method that returns a copy of TOPLEVEL_BINDING. The @src instance variable carries the content of a code tag, and the @safe_level instance variable contains the safe level required by the user. If no safe level is set, the content of the tag is simply evaluated. Otherwise, ERB builds a quick *Sandbox (149)*: it makes sure that the global safe level is exactly what the user asked for and